

FACULDADE DE TECNOLOGIA DO ESTADO DE SÃO PAULO
DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

ARTEFATOS DO PROJETO DE SOFTWARE
**CLASSIFICAÇÃO DE MANGANÊS E COBRE NA FOLHA DA MEXERICA,
ORIENTADO POR REDES NEURAIS**

Amanda Vithória Alves Freitas
Juliano Lopes Rodrigues Sales
Lucas Gomes Fagundes
Valéria de Freitas

Conteúdo

1	CANVAS: Modelo de Negócio e Estratégia	2
2	SWOT: Análise Estratégica	3
3	UX/UI: Design de Interface e Experiência do Usuário	4
4	Website: Estrutura e Especificações Técnicas Web	5
5	Diagramas UML: Modelagem Visual do Sistema	6
6	DevOps: Infraestrutura, Automação e Entrega Contínua	9
	Conclusão	13
	Referências	14

1 CANVAS: Modelo de Negócio e Estratégia

O Business Model Canvas é uma ferramenta de modelagem estratégica que permite descrever, analisar e projetar o modelo de negócio de forma clara e objetiva. O Canvas do projeto de classificação de manganês e cobre na folha da mexerica contempla os principais blocos de valor do sistema, focado na análise de deficiências nutricionais em folhas de mexerica.

Proposta de Valor

O sistema ajuda agricultores a identificarem deficiências de manganês e cobre na folha da mexerica, oferecendo análises automatizadas por meio de imagens e inteligência artificial.

Segmento de Mercado

O público-alvo são agricultores de mexerica do Vale do Ribeira, com foco em produtores rurais, horticultores e outros interessados na melhoria da produtividade agrícola.

Relacionamento com o Cliente

O relacionamento será realizado por WhatsApp, aplicativo móvel e site institucional, com feedback automatizado de análises por meio de gráficos e mapas.

Canais

A divulgação e acesso ao sistema ocorrerão via secretarias de agricultura municipais, site, e-mail e WhatsApp, fortalecendo a comunicação direta com o público-alvo.

Atividades-Chave

As principais atividades são identificar deficiências nutricionais, oferecer suporte técnico e manter o sistema.

Recursos-Chave

O sistema depende de equipe técnica, hospedagem para site e banco de dados, e infraestrutura de análise de imagens.

Parcerias-Chave

As parcerias envolvem associações de fazendeiros, secretarias de agricultura e participação em feiras do setor.

Estrutura de Custos

Inclui manutenção de equipamentos, aluguel de espaço físico, compra de materiais e custos de servidores.

Fontes de Renda

A monetização pode ocorrer por meio de aluguel do sistema, cobrança por equipamentos e serviços de instalação e manutenção.

Figura 1: Modelo de Negócios Canvas



2 SWOT: Análise Estratégica

A Análise SWOT (Strengths, Weaknesses, Opportunities, Threats) ou FOFA (Forças, Oportunidades, Fraquezas e Ameaças) [1] é uma ferramenta fundamental para planejamento estratégico¹ que permite identificar fatores internos e externos que impactam o projeto. A análise SWOT foi realizada com o objetivo de compreender os principais pontos fortes, fracos, oportunidades e ameaças relacionadas ao projeto de classificação de manganês e cobre na folha da mexerica.

2.1 Matriz SWOT

A matriz SWOT do projeto está apresentada na Tabela 1, organizando os fatores internos (forças e fraquezas) e externos (oportunidades e ameaças).

¹A análise SWOT foi criada na década de 1960 por Albert Humphrey no Stanford Research Institute.

Tabela 1: Análise SWOT do Projeto

FATORES INTERNOS	
FORÇAS (Strengths)	FRAQUEZAS (Weaknesses)
• É alinhado com iniciativas sustentáveis. • Melhora a eficiência e rapidez no diagnóstico de doenças. • Reduz o risco de pés de mexerica ainda saudáveis. • Reduz o desperdício de alimentos saudáveis. • É alinhado com os objetivos da ODS.	• Poucos estudos na área envolvendo plantas cítricas. • O treinamento do modelo de IA depende de um grande banco de dados. • O diagnóstico depende da qualidade da imagem. • Dificuldade de acesso à internet por parte dos produtores.
FATORES EXTERNOS	
OPORTUNIDADES (Opportunities)	AMEAÇAS (Threats)
• Expandir para outras deficiências como: zinco, ferro etc. • Parcerias com empresas, cooperativas e universidades para divulgar o produto. • Crescente demanda por agricultura de precisão.	• Resistência por parte dos produtores tradicionais. • Concorrência com outras soluções, como drones, por exemplo.

Esse tipo de avaliação permite identificar elementos que podem impactar diretamente o sucesso do sistema, além de apontar caminhos para melhorias contínuas e decisões estratégicas.

2.2 Recomendações Estratégicas

Com base na análise SWOT, as principais estratégias devem focar em:

- **Forças + Oportunidades:** Posicionar a solução no mercado de agricultura de precisão, destacando o alinhamento com práticas sustentáveis e os Objetivos de Desenvolvimento Sustentável (ODS), e buscar parcerias com cooperativas e universidades para ampliar a divulgação.
- **Forças + Ameaças:** Reforçar a proposta de valor frente à concorrência (como drones), enfatizando o diagnóstico rápido por imagem da folha, a redução de desperdício e a preservação de pés saudáveis de mexerica.
- **Fraquezas + Oportunidades:** Estabelecer parcerias com universidades e cooperativas para ampliar o banco de dados de treinamento da IA e validar cientificamente a solução em plantas cítricas, aproveitando a demanda por agricultura de precisão.
- **Fraquezas + Ameaças:** Mitigar a resistência dos produtores tradicionais com materiais de capacitação sobre captura de imagens e uso do sistema, e planejar funcionalidades que reduzam a dependência de conectividade contínua.

3 UX/UI: Design de Interface e Experiência do Usuário

Esta seção apresenta um Mockup de alta fidelidade da versão Mobile e Web do sistema, além de um guia de estilos com a paleta de cores utilizada no projeto. O design foi pensado para ser

intuitivo, acessível e alinhado com as necessidades dos usuários finais.

3.1 Mockup Interface



Figura 2: Mockup de alta fidelidade

3.2 Guia de Estilos

Paleta de cores principais do Design System do projeto:

Tabela 2: Paleta de Cores do Projeto

Elemento	Cor	Código HEX	Uso
Primária		#58B741	Botões principais
Secundária		#FF9E00	Elementos secundários, destaques
Fundo		#FAF1E5	Background padrão
Texto		#000000	Texto principal
Sucesso		#34C759	Mensagens de sucesso
Erro		#E91F23	Mensagens de erro

4 Website: Estrutura e Especificações Técnicas Web

O mapa do site (sitemap) mostra a hierarquia e organização das páginas do website da equipe:

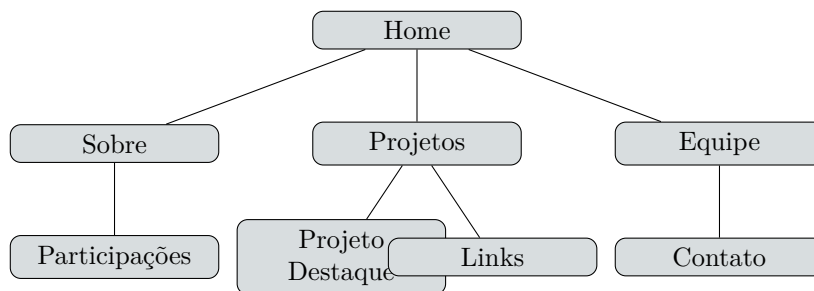


Figura 3: Estrutura de navegação do website da equipe

4.1 Conteúdo das Páginas

Páginas principais do site da equipe:

- **Home:** Apresentação da marca Vitalliz
- **Sobre:** História da equipe, missão, visão e valores
- **Projeto em Destaque:** Showcase do projeto desenvolvido com descrições e imagens
- **Equipe:** Perfil dos membros (nome, foto, função, LinkedIn/GitHub)

5 Diagramas UML: Modelagem Visual do Sistema

A modelagem UML permite representar visualmente a arquitetura e as interações do sistema. A seguir, são apresentados os principais diagramas que suportam o projeto de classificação de manganês e cobre na folha da mexerica.

5.1 Diagrama de Casos de Uso

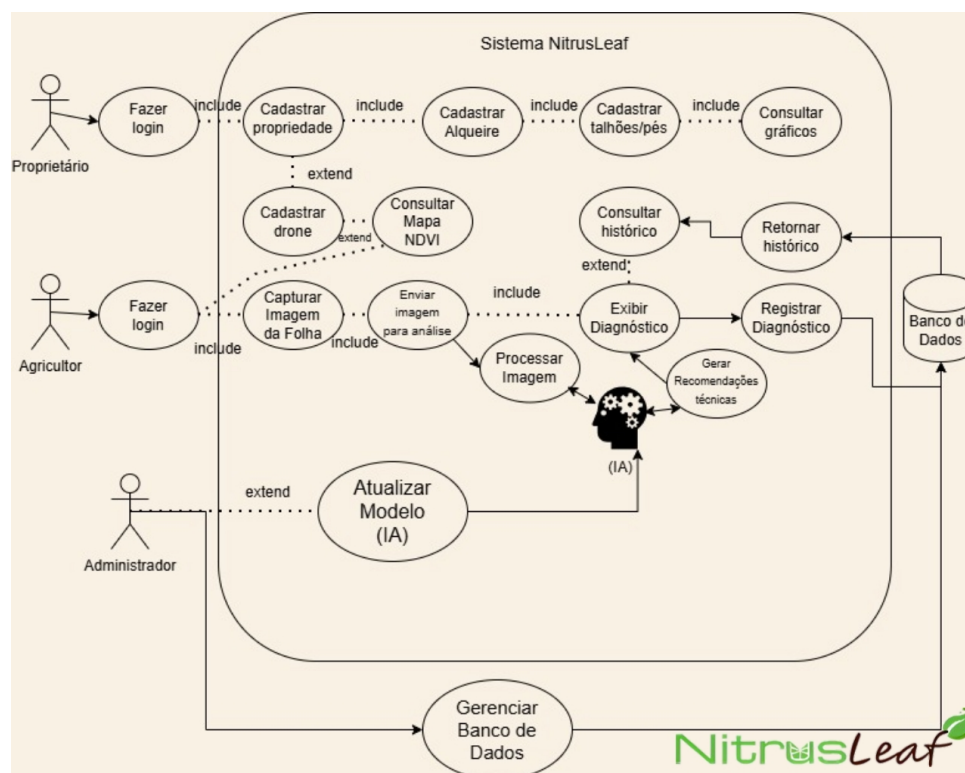
Atores do Sistema:

- **Usuário (Cliente):** acessa o sistema para realizar cadastro/login, consultar informações, criar/editar solicitações e acompanhar status.
- **Administrador:** gerencia cadastros, permissões, configurações gerais e monitora registros/relatórios.
- **Operador/Atendente:** valida dados enviados pelos usuários, aprova/reprova solicitações e atualiza status operacionais.
- **Sistema Externo (API/Integrações):** provê dados de serviços de terceiros (ex.: pagamento, mapas, notificações).

Casos de Uso do Sistema:

- **Autenticar Usuário:** cadastro, login e recuperação de senha.
- **Gerenciar Perfis:** editar dados pessoais, preferências e documentos.
- **Registrar Solicitação:** criar, editar e cancelar solicitações.
- **Acompanhar Status:** visualizar andamento, receber notificações e histórico.
- **Gerir Operações (Backoffice):** triagem, aprovação e ajustes operacionais.
- **Relatórios e Auditoria:** visualizar métricas, exportar dados e rastrear logs.
- **Integrações:** consultar/enviar dados para APIs externas.

Figura 4: Diagrama de Caso de Uso

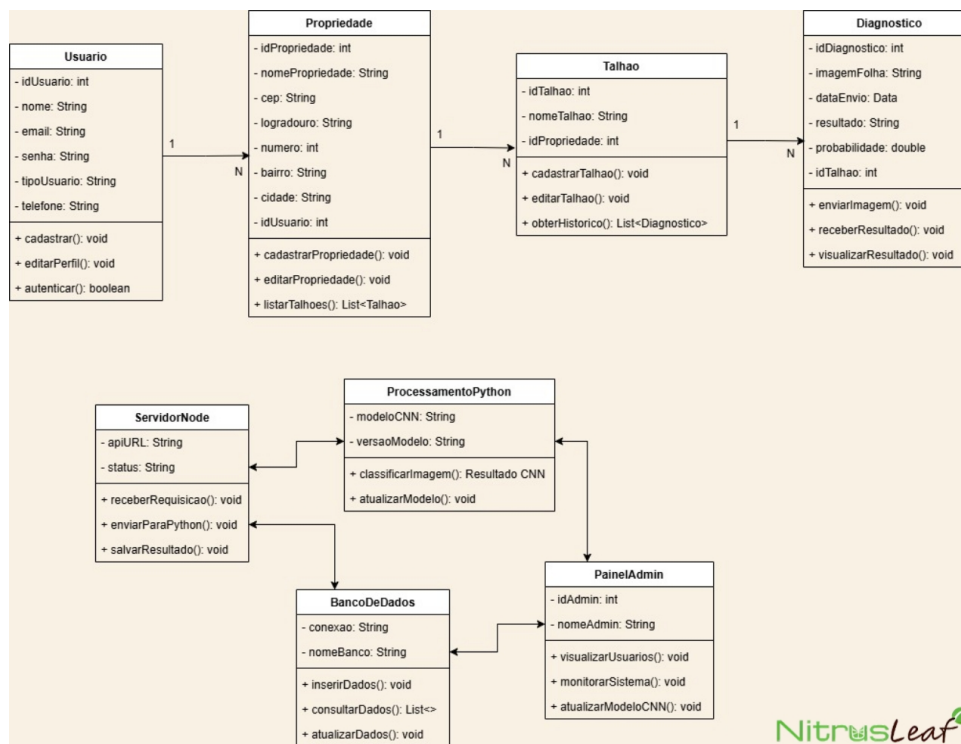


5.2 Diagrama de Classe

O diagrama de classes resume as principais entidades do sistema e seus relacionamentos.

- **Propriedade:** cadastra e gerencia propriedades rurais vinculadas a um usuário.
- **Usuário:** mantém dados pessoais e de autenticação, suportando cadastro e login.
- **Talhão:** subárea da propriedade que guarda metadados e histórico de diagnósticos.
- **Diagnóstico:** registra análise por imagem, vínculo ao talhão e resultado do laudo.
- **ServidorNode:** faz a ponte do frontend com a API e encaminha requisições para processamento.
- **ProcessamentoPython:** executa classificação de imagens com CNN e atualiza o modelo.
- **BancoDeDados:** gerencia conexão e operações CRUD para persistência dos dados.
- **PainelAdmin:** área administrativa para gestão de usuários, modelos e monitoramento.

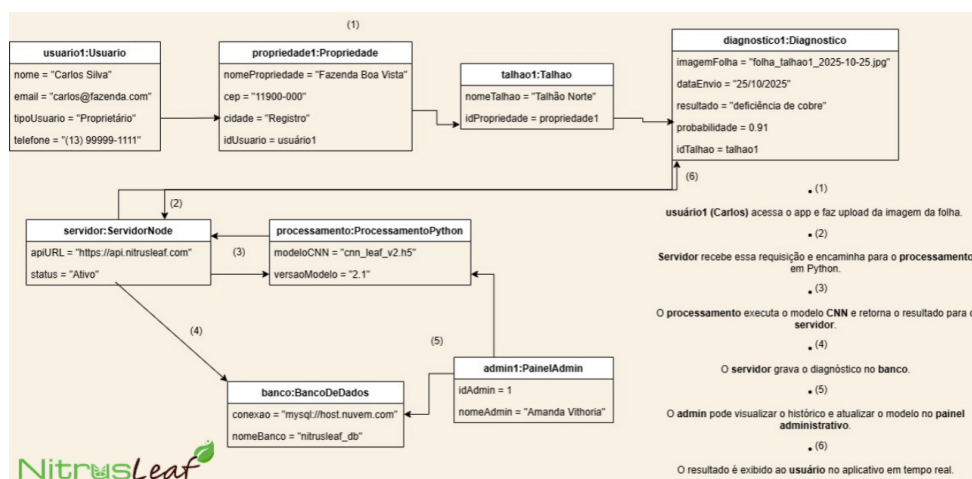
Figura 5: Diagrama de Classe



5.3 Diagrama de Objetos

O diagrama de objetos apresenta instâncias concretas e o fluxo de criação e vinculação entre elas.

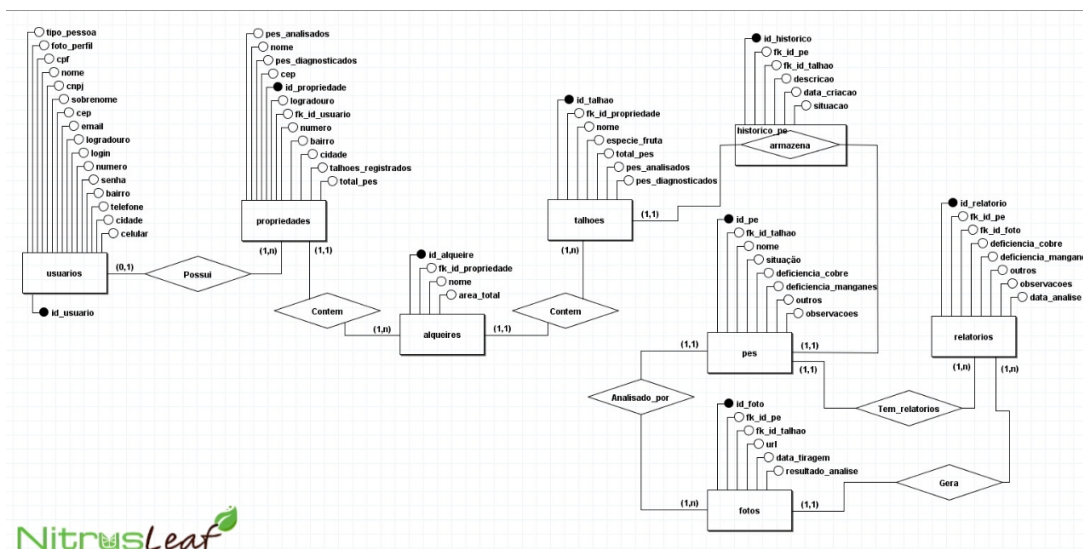
Figura 6: Diagrama de Objetos



5.4 Diagrama Entidade-Relacionamento

O diagrama Entidade-Relacionamento descreve a estrutura lógica do banco de dados, com entidades como Usuários, Propriedades, Talhões, Pés, Fotos e Relatórios.

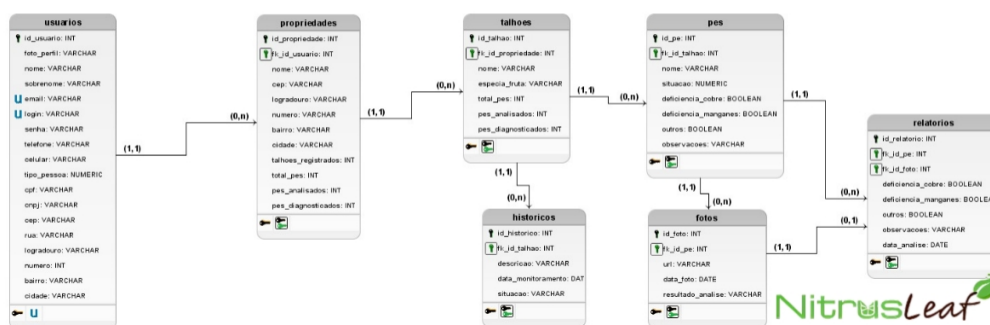
Figura 7: Diagrama Entidade-Relacionamento



5.5 Diagrama Modelo Lógico

O modelo lógico descreve a hierarquia dos dados e a ligação entre entidades do sistema.

Figura 8: Diagrama Modelo Lógico



6 DevOps: Infraestrutura, Automação e Entrega Contínua

DevOps [2] é uma cultura e conjunto de práticas que integra desenvolvimento de software (Dev) e operações de TI (Ops), focando em automação, colaboração e entrega contínua. No *Nitrus-Leaf Mobile*, o repositório Vitalliz/nitrusleaf-mobile adota Git, GitHub Actions, backend Supabase e aplicativo Expo (React Native), com validação automática a cada alteração enviada ao repositório.

6.1 Pipeline CI/CD

O pipeline de integração e entrega contínua executa três *jobs* no GitHub Actions (Build, Test e Deploy) a cada *push* em qualquer branch ou em *pull request* direcionado à main. A Figura 9 resume o fluxo; a etapa **Monitor** representa a observação do resultado (painel do GitHub Actions e Supabase Dashboard) e o retorno de correções ao código — ciclo de melhoria contínua.

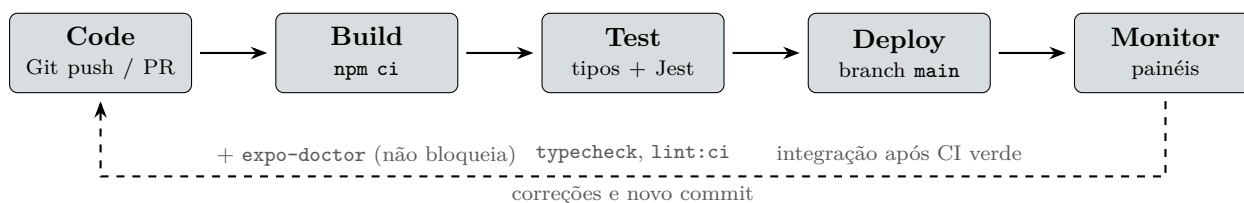


Figura 9: Pipeline CI/CD implementado no repositório (.github/workflows/ci.yml)

Correspondência entre etapas e jobs

Tabela 3: Etapas do pipeline e implementação no GitHub Actions

Etapa	Job	Comandos / comportamento real
Code	—	<i>Push</i> ou PR; código em nitrusleaf-mobile/
Build	build	npm ci; npx expo-doctor (continue-on-error)
Test	test	npm run typecheck; npm test -- --ci; npm run lint:ci
Deploy	deploy	Somente <i>push</i> na main; confirma integração (EAS/Supabase opcionais)
Monitor	—	GitHub Actions (status do workflow); Supabase Dashboard (API, Auth, BD)

O ambiente de execução utiliza **Node.js 22** no runner do GitHub Actions.

6.2 Exemplo de Configuração CI/CD

Trecho conceitual equivalente ao arquivo .github/workflows/ci.yml:

```

name: CI
on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
      - run: npm ci
      - run: npx expo-doctor
        continue-on-error: true

  test:
    needs: build
    steps:
      - run: npm run typecheck
      - run: npm test -- --ci
      - run: npm run lint:ci

  deploy:
  
```

```

needs: test
if: branch == main
steps:
  - run: echo "CI verde na main"

```

Comandos locais:

- `npm run validate` — verificação de tipos (`tsconfig.ci.json`) + testes Jest;
- `npm run lint:ci` — ESLint na camada core (`utils`, `repositories`, `services`, `contexts`);
- `npm run lint` — análise completa do projeto (inclui telas; pode reportar avisos legados).

6.3 Infraestrutura como Código

Tabela 4: Ferramentas DevOps utilizadas no NitrusLeaf Mobile

Categoria	Ferramenta	Finalidade
Controle de Versão CI/CD Backend (BaaS)	Git + GitHub GitHub Actions Supabase	<code>main</code> e branches da equipe via PR Jobs <code>build</code> , <code>test</code> e <code>deploy</code> PostgreSQL, Auth JWT e API REST
App Mobile Persistência local Testes Qualidade	Expo SDK 54 + React Native <code>expo-sqlite</code> Jest + <code>jest-expo</code> ESLint (<code>lint:ci</code>)	iOS, Android e Web SQLite no dispositivo (offline) Testes em <code>src/utils/__tests__</code> Validação automática da camada core
Configuração	<code>.env</code> / <code>.env.example</code>	Chaves Supabase (não versionadas)

6.4 Infraestrutura do Sistema

Componentes por ambiente

Tabela 5: Componentes de infraestrutura

Componente	Descrição
Desenvolvimento local	• Node.js ≥ 20 (CI usa 22) • Variáveis em <code>.env</code> conforme <code>.env.example</code> • Branches dos membros integradas à <code>main</code> via pull request com CI verde
Nuvem (Supabase)	• PostgreSQL gerenciado, autenticação e API REST • HTTPS entre app e backend • Painel para logs, usuários e consulta ao banco (etapa Monitor)
Dispositivos móveis	• Binário/servidor de desenvolvimento via Expo • <code>expo-sqlite</code> para cache local • Camada <code>src/repositories</code> para acesso local e remoto
Segurança	• TLS nas requisições à API • Sessão JWT via Supabase Auth • Chave <code>anon</code> no cliente; <code>.env</code> ignorado pelo Git

Fluxo de dados

- **Usuário mobile:** autentica no Supabase Auth e consome dados via API.
- **Banco remoto:** PostgreSQL no Supabase.
- **Cache local:** SQLite (`expo-sqlite`) no aparelho.
- **CI/CD:** cada *push* dispara build e testes; merge na `main` após pipeline bem-sucedido.
- **Monitoramento:** status dos workflows no GitHub; métricas e saúde da API no Supabase Dashboard.

6.5 Arquitetura de Deploy

A Figura 10 separa validação do código (CI), execução do app e backend em nuvem. O GitHub Actions não hospeda o aplicativo: ele *valida* o repositório antes da integração na `main`.

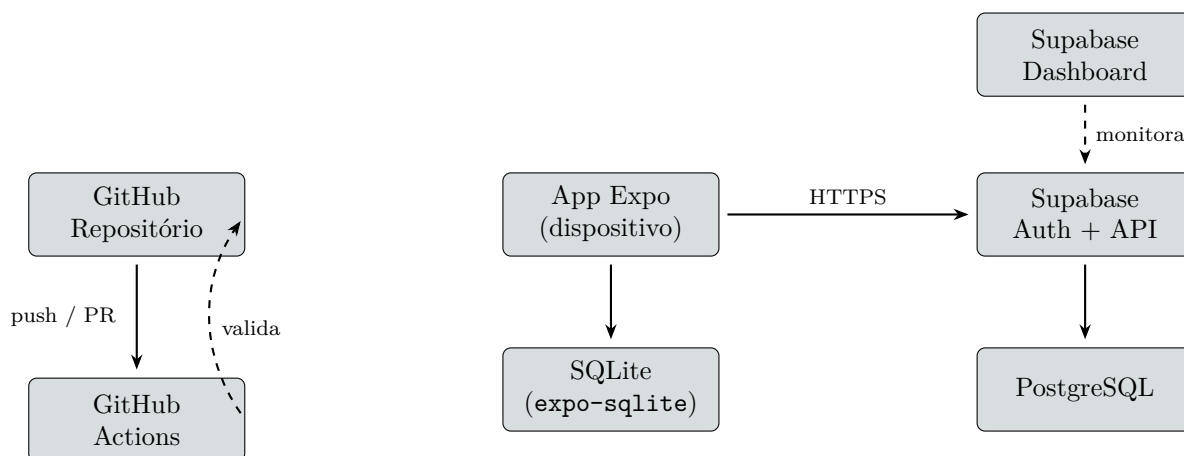


Figura 10: Arquitetura de deployment e monitoramento

6.6 Métricas e SLA

Indicadores acompanhados na fase atual do projeto:

- **Disponibilidade:** conforme plano Supabase (API e banco gerenciados).
- **Pipeline CI:** execução automática em *push* e PR; jobs `build` e `test` obrigatórios.
- **Testes:** suíte Jest em `src/utills`; comando `npm run validate` localmente.
- **Qualidade:** `lint:ci` bloqueante no workflow; `npm run lint` ampliado para desenvolvimento.
- **Deploy frequency:** integração contínua via merge na `main` após CI verde.
- **Lead time:** revisão em PR com checks do GitHub antes do merge.
- **MTTR:** correção versionada, novo *push* e reexecução automática do pipeline.

Conclusão

Este documento apresenta os principais artefatos do projeto **Classificação de Manganês e Cobre na Folha da Mexerica**, reunindo a documentação necessária para o desenvolvimento, a modelagem e a entrega do software *NitrusLeaf Mobile*.

Os artefatos documentados incluem:

- **CANVAS:** Modelo de negócio e estratégia
- **SWOT:** Análise estratégica de forças, fraquezas, oportunidades e ameaças
- **UX/UI:** Design de interface e experiência do usuário
- **Website:** Estrutura e especificações técnicas web
- **Diagramas UML:** Modelagem visual do sistema
- **DevOps:** Infraestrutura, automação e práticas de entrega contínua

Referências

Referências

- [1] OLIVEIRA, P.; COSTA, M. Machine learning applications in dynamic systems. *IEEE Transactions on Systems, Man, and Cybernetics*, IEEE, v. 53, n. 8, p. 4567–4580, 2023.
- [2] WANG, Y.; ZHANG, H.; LEE, K. Comparative study of numerical integration methods. *Numerical Algorithms*, Springer, v. 92, n. 4, p. 1523–1547, 2023.